TECHNICAL REPORT · ITUC201 · COMPUTER NETWORKS · ADVANCED INVESTIGATION

HTTP Keep-Alive versus New-Connection Requests

Student	Kismat Shah	Student ID	25cs099
Class	CSE-2	Date	25 October 2026
Live website	reuse-reconnect.netlify.app	Source code	github.com/kismats1107-stack/reuse-reconnect
Video	YouTube link (Unlisted) given in the submission form		

1 Objective

To find out, by modelling and by real measurement, how much time a web page saves when HTTP reuses one TCP connection for all its files (**keep-alive / persistent connections**) instead of opening a **new TCP connection for every file** (`Connection: close`), and to explain the difference packet by packet.

- Build an interactive 3D visualisation that races the two strategies side by side with every SYN, GET, response and FIN visible.
- Derive a simple timing model and check it against a real HTTP server and client, with the traffic captured in Wireshark.
- Show how the saving depends on the number of files *N* and the round-trip time *RTT*.

2 Essential theory

TCP connection set-up and teardown

TCP (RFC 9293) must open a connection before any data flows. The **three-way handshake** is SYN → SYN-ACK → ACK. The client can send its HTTP request together with the final ACK, so the handshake costs the client **one round-trip time (1 RTT)** before the first request leaves. Closing uses FIN / ACK in each direction; it does not delay the page, but each closed connection leaves a socket in TIME_WAIT and costs the server work.

Non-persistent versus persistent HTTP

A request/response pair costs one further RTT (plus the server time and the time to transmit the file). With a **new connection per file**, every file pays a handshake *and* a request: 2 RTT each. With **keep-alive**, the default in HTTP/1.1 (RFC 9112 §9.3), the handshake is paid once and every later file needs only 1 RTT. The `Connection: close` header (RFC 9110 §7.6.1) asks for the connection to be closed after the response, which turns HTTP back into one-connection-per-file.

For *N* files fetched one after another (no pipelining, no parallel connections), with Σ = total server and transfer time:

$$T_{\text{new connection}} = N \cdot 2{\cdot}RTT + \Sigma$$
$$T_{\text{keep-alive}} = RTT + N \cdot RTT + \Sigma$$
$$\text{Saving} = T_{new} - T_{keep} = (N - 1) \cdot RTT \quad \text{handshakes: } N \to 1$$

The saving grows linearly with both the number of files and the distance to the server; for a single file there is no difference. Relative saving tends to 50% as N grows, because keep-alive needs about half as many round trips.

3 Steps followed (method)

Part A: the interactive model (website)

1. Wrote a discrete-event simulation (`src/lib/simulation.ts`) that emits every packet (SYN, SYN-ACK, ACK+GET, response, FIN) for both strategies with link speed 100 Mbps and 1 ms server time per file. Its totals match the formulas above; 82 automated tests (Vitest) check the timings and counts.
2. Rendered the two strategies as two lanes of a 3D world (React Three Fiber). Packets fly between a browser and a server building; lane A (orange) opens a new connection per file, lane B (green) reuses one. A scoreboard counts connections, handshakes, files and time.
3. Added controls for the number of files (1–40) and server distance (0, 10, 50, 100 ms RTT), a guided tour with a voice narrator, and a Results page with 3D towers, charts, sequence diagrams and the real measurements.

Part B: the real experiment (Python + Wireshark)

1. `lab/server.py` : an HTTP/1.1 server (Python `http.server` , threaded) that serves the same file set as the website and counts the connections it accepts.
2. `lab/client.py` : a raw-socket HTTP client (TCP_NODELAY, no HTTP library) that fetches N files either over one keep-alive connection or with a new connection and `Connection: close` for each file, timing from the first `connect()` to the last response byte.
3. `lab/relay.py` : since everything runs on one laptop (real RTT ≈ 0), a relay adds RTT/2 in each direction and holds the first request on a new connection for 1 RTT to emulate the handshake over distance.
4. `lab/capture.py` : Dumpcap (Wireshark 4.6.8) captures the loopback adapter; tshark counts SYN packets per trial.
5. `lab/run_experiment.py` ran 5 page sizes × 4 distances × 2 strategies × 5 repetitions = **200 trials** on Windows 11, Python 3.14.6, and wrote `measurement-20261022-203701.json` and `capture-20261022-203701.pcapng` . Each trial counts connections three independent ways: client sockets, server accepts and captured SYN packets.

4 Screenshots and results



Figure 1. The 3D lab after a race with 10 files and a server 50 ms away. Lane A used 10 connections and 10 handshakes and finished in 1042 ms; lane B used 1 connection and finished in 592 ms, so keep-alive won by 450 ms (43%).

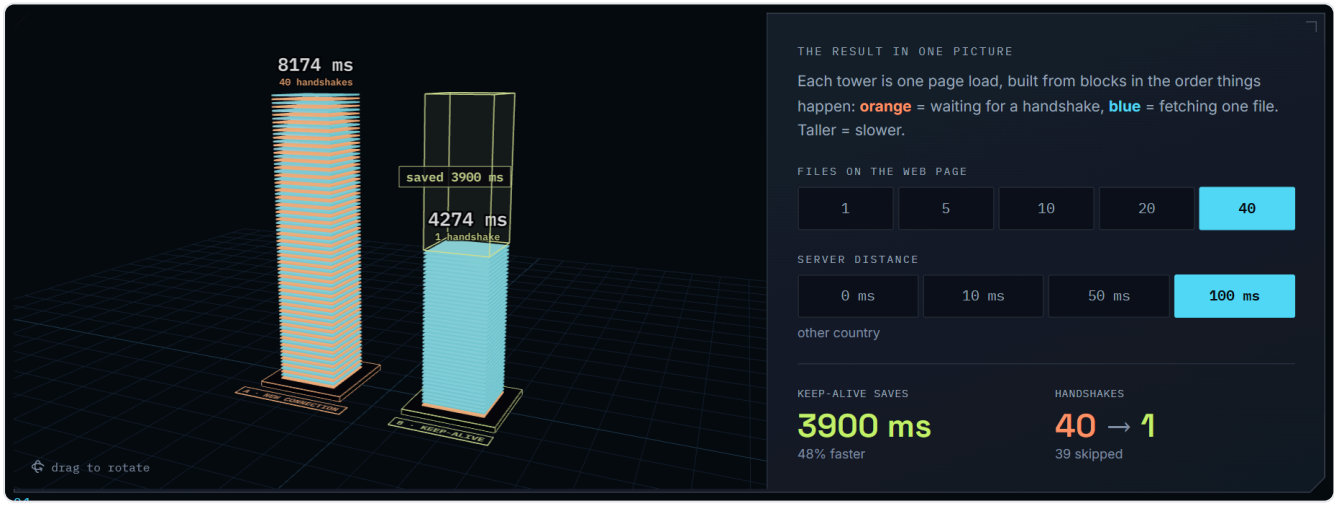


Figure 2. Results page, 40 files at 100 ms. Each tower is one page load built from its events: orange = waiting for a handshake, blue = fetching a file. Keep-alive saves 3900 ms (48%) and skips 39 of 40 handshakes.

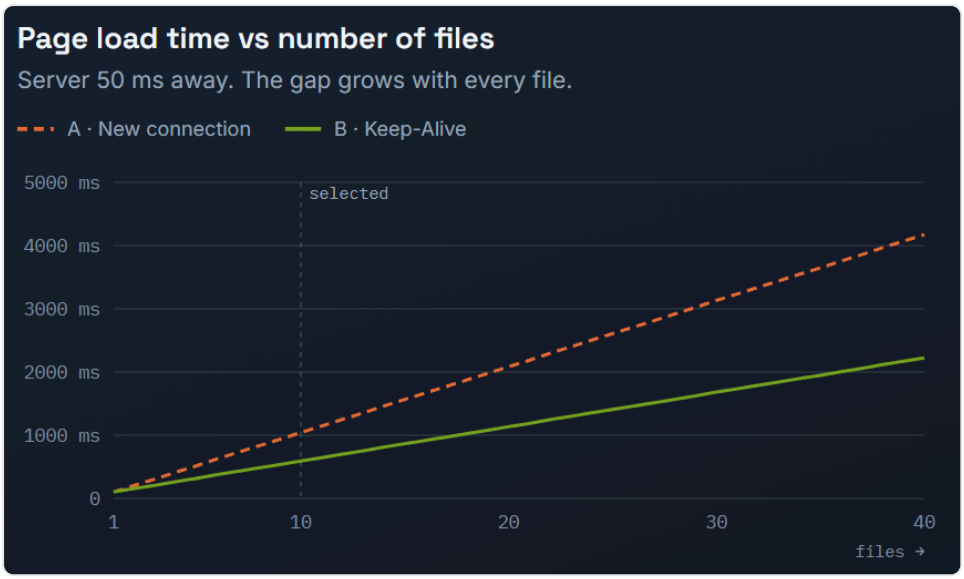


Figure 3. Model: page load time against number of files at 50 ms. The gap widens by one RTT with every extra file.

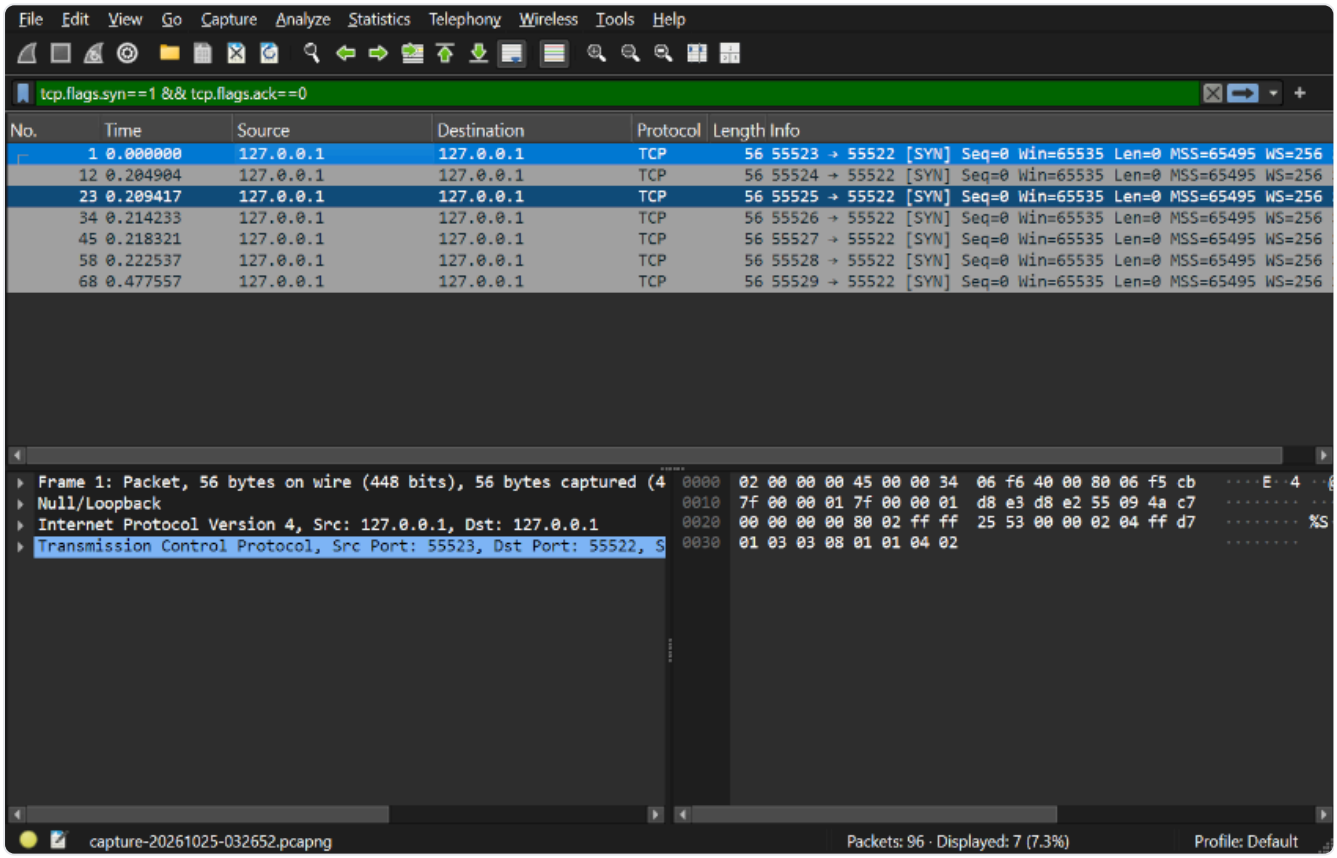


Figure 4. Wireshark on a capture of the lab (5 files, both strategies) with the display filter `tcp.flags.syn==1 && tcp.flags.ack==0`, which shows one packet per new TCP connection. "Displayed: 7" = 1 warm-up + 5 for new-connection + 1 for keep-alive.



Figure 5. Sequence diagrams for 3 files at 50 ms RTT, drawn on the same time scale. Left: every file opens and closes its own connection (#1, #2, #3), loaded at 309 ms. Right: one connection carries all three GETs, loaded at 209 ms. The difference is exactly two handshakes: (3 − 1) × 50 ms = 100 ms.

Real measurements



Figure 6. Results page, “Real proof”: dots are measured medians, dashed lines are the model, at 50 ms RTT. The measured points sit on the model lines.

RTT	Files N	New connection (ms)	Keep-alive (ms)	Measured saving (ms)	(N−1)·RTT (ms)	Connections A / B
0 ms · plain loopback (no added delay)						
	1	3.0	4.0	-1.1	–	1 / 1
	5	17.4	7.1	10.4	–	5 / 1
	10	34.6	11.4	23.2	–	10 / 1
	20	62.4	14.5	47.8	–	20 / 1
	40	93.3	18.8	74.4	–	40 / 1
10 ms · emulated by the relay						
	1	27.9	28.0	-0.1	0	1 / 1
	5	141	82.8	58.4	40	5 / 1
	10	285	157	129	90	10 / 1
	20	559	303	256	190	20 / 1
	40	1,128	578	550	390	40 / 1
50 ms · emulated by the relay						
	1	109	107	2.3	0	1 / 1
	5	544	325	219	200	5 / 1
	10	1,084	593	491	450	10 / 1
	20	2,169	1,134	1,035	950	20 / 1
	40	4,342	2,240	2,102	1,950	40 / 1
100 ms · emulated by the relay						
	1	210	209	1.1	0	1 / 1
	5	1,049	631	418	400	5 / 1
	10	2,091	1,148	943	900	10 / 1
	20	4,189	2,201	1,988	1,900	20 / 1
	40	8,359	4,294	4,065	3,900	40 / 1

Table 1. Median page-load time of 5 repetitions per cell (200 trials in total). Connections are the SYN packets counted in the capture; the client and server counts were identical. In **200 of 200** trials all three counters agreed with the expected value (N for new connection, 1 for keep-alive).

5 Calculations

Worked example (model, N = 10, RTT = 50 ms). Σ (server + transfer of the 10 files at 100 Mbps) = 42 ms.
T_{new} = 10 × 2 × 50 + 42 = **1042 ms** · T_{keep} = 50 + 10 × 50 + 42 = **592 ms**
Saving = (10 − 1) × 50 = **450 ms** = 450 / 1042 = **43%** faster; handshakes 10 → 1.

Model against measurement.

- 10 files, 50 ms: model 1042 / 592 ms; measured 1,084 / 593 ms (within 4%).
- 40 files, 100 ms: model 8174 / 4274 ms; measured 8,359 / 4,294 ms (within 2%).
- 20 files, 50 ms: measured saving 1,035 ms against (20 − 1) × 50 = 950 ms predicted; the extra 85.2 ms ≈ 4.5 ms per extra connection is the real cost of creating and closing a socket, which the model leaves out.
- 40 files, 0 ms (no network delay at all): 93.3 vs 18.8 ms. Keep-alive is still 5.0× faster, so each new connection costs about 1.9 ms of operating-system work even on loopback.

6 Interpretation

- **The model is confirmed.** Over all distances the measured saving tracks (N − 1)·RTT and the measured times lie within a few percent of the model. The connection counts (N against 1) match in all 200 trials on the client, the server and the wire.
- **Distance matters more than bandwidth.** The files are small, so transfer time is a few milliseconds; the round trips dominate. At 100 ms the 40-file page drops from about 8.4 s to 4.3 s.
- **One file: no benefit.** With N = 1 both strategies need one handshake, and the measured times are equal within noise (e.g. 109 vs 107 ms at 50 ms).
- **The saving is slightly bigger than the model.** Socket creation, the FIN exchange and server accept work add about 1.9–4.5 ms per extra connection on this laptop.
- **Limits.** Requests were sequential: no pipelining, no parallel connections, no TLS and no slow-start effects, and RTT was emulated on one machine. Real browsers open about 6 parallel connections, which hides some of the cost, and HTTPS adds 1–2 more RTT per new connection for the TLS handshake, which makes keep-alive even more valuable.

> **Conclusion.** Reusing one TCP connection removes one handshake round trip for every file after the first. For a typical page of many small files on a distant server this almost halves the load time, which is why persistent connections are the default in HTTP/1.1.

7 Exam-related answer

Question (Kurose & Ross style). A web page has a base HTML file that references 10 small objects. The RTT between the client and the server is 50 ms, and transmission times are negligible. How long does it take to get the whole page with (a) non-persistent HTTP and no parallel connections, (b) non-persistent HTTP with 5 parallel connections, (c) persistent HTTP without pipelining, (d) persistent HTTP with pipelining?

Answer. The base file always costs 2 RTT (handshake + request).

a. Each of the 10 objects needs a new connection: 2 RTT + 10 × 2 RTT = **22 RTT = 1100 ms**.

b. The objects are fetched in 2 rounds of 5 parallel connections: 2 RTT + 2 × 2 RTT = **6 RTT = 300 ms**.

c. The connection is reused, 1 RTT per object: 2 RTT + 10 × 1 RTT = **12 RTT = 600 ms**.

d. All 10 requests are sent back to back: 2 RTT + 1 RTT = **3 RTT = 150 ms**.

(a) − (c) = 10 RTT = 500 ms. This matches this project's formula with N = 11 files in total: (N − 1) · RTT = 10 × 50 = 500 ms, and is what lane A against lane B shows in the 3D lab.

8 References

1. R. Fielding, M. Nottingham, J. Reschke (eds.), *RFC 9112: HTTP/1.1*, IETF, June 2022, §9 "Connection Management", §9.3 "Persistence".
2. R. Fielding, M. Nottingham, J. Reschke (eds.), *RFC 9110: HTTP Semantics*, IETF, June 2022, §7.6.1 "Connection".
3. W. Eddy (ed.), *RFC 9293: Transmission Control Protocol (TCP)*, IETF, August 2022, §3.5 "Establishing a Connection".
4. J. F. Kurose, K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021, §2.2.2 "Non-Persistent and Persistent Connections".
5. MDN Web Docs, "Connection management in HTTP/1.x", developer.mozilla.org.
6. Wireshark Foundation, *Wireshark User's Guide* and *dumpcap / tshark manual pages*, wireshark.org.
7. Python Software Foundation, *http.server* and *socket* module documentation, docs.python.org.

9 Instructions to reproduce

Run the website

Open reuse-reconnect.netlify.app, or run it locally (Node.js 20+):

```
git clone https://github.com/kismats1107-stack/reuse-reconnect
cd reuse-reconnect          # or unzip reuse-reconnect-source.zip
npm install
npm run dev                 # http://localhost:5180  ·  #/lab and #/results
npm test                    # 82 tests of the model
```

Repeat the real measurement

Needs Python 3.10+ (standard library only). Wireshark with Npcap and its loopback adapter is optional: without it the trials run and are timed, but the packets are not captured.

```
python lab/run_experiment.py                # full sweep: 1–40 files × 0–100 ms, 5 reps
python lab/run_experiment.py --objects 10 --rtt 50
python -m unittest lab/test_lab.py -v       # lab self-tests
```

Results are written to `lab/out/measurement-<time>.json` and `lab/out/capture-<time>.pcapng`. To see the handshakes, open the capture in Wireshark and apply the display filter `tcp.flags.syn==1 && tcp.flags.ack==0`: there is one SYN per TCP connection. Copy the JSON to `src/data/measurement.json` to show it on the Results page.

Supporting files

File	Contents
`reuse-reconnect-source.zip`	All editable source: website, Python lab, tests, video tooling, this report's generator
`measurement-20261022-203701.json`	Raw data of the 200 trials (every sample, medians, quartiles, counters)
`capture-20261022-203701.pcapng`	Wireshark packet capture of the same run
`Technical-Report.pdf`	This report (rebuilt with `npm run report`)